1. Which built-in function is used to open files and return a file stream object in Python?

(A) file()
(B) open()
(C) read()
(D) fopen()

2. What is the default file opening mode if the mode argument is omitted in open('data.txt')?

(A) 'w' (write)
(B) 'r' (read text mode)
(C) 'a' (append)
(D) 'rb' (read binary)

3. Which file mode truncates an existing file to zero length or creates a new file for writing?

(A) 'r'
(B) 'w'
(C) 'a'
(D) 'x'

4. Which file mode opens a file for writing without truncating, appending all new data at the end?

(A) 'r'
(B) 'w'
(C) 'a'
(D) 'r+'

5. Which file mode opens a file for exclusive creation, failing if the file already exists?

(A) 'w'
(B) 'a'
(C) 'x'
(D) 'r+'

6. Which statement guarantees that a file is closed immediately when execution leaves the block, even if exceptions occur?

(A) try-except statement
(B) with statement (Context Manager)
(C) finally block only
(D) close() method

7. Which file method reads the entire remaining content of a file as a single string?

(A) readline()
(B) read()
(C) readlines()
(D) readall()

8. Which file method reads a single line terminated by a newline character '\n'?

(A) read()
(B) readline()
(C) readlines()
(D) peek()

9. Which file method reads all lines from a file and returns them as a list of strings?

(A) read()
(B) readline()
(C) readlines()
(D) listfile()

10. What is the most memory-efficient way to iterate through a massive multi-gigabyte text file line-by-line?

(A) for line in file.readlines():
(B) for line in file:
(C) content = file.read().splitlines()
(D) while file.read():

11. Which method writes a string to a file without automatically adding a trailing newline character?

(A) write()
(B) writeline()
(C) print()
(D) put()

12. Which method writes a sequence (list) of strings to a file?

(A) write_all()
(B) writelines()
(C) writelist()
(D) dump()

13. Which method moves the file stream read/write pointer to a specified byte offset?

(A) seek(offset, whence)
(B) tell()
(C) move()
(D) jump()

14. Which method returns the current integer byte offset of the file pointer?

(A) seek()
(B) tell()
(C) pos()
(D) cursor()

15. Which standard library module provides operating system dependent functionality like file paths and directories?

(A) sys
(B) os
(C) file
(D) platform

16. Which module introduced in Python 3.4 provides an object-oriented filesystem path abstraction?

(A) os.path
(B) pathlib
(C) filepath
(D) glob

17. What is Object Serialization in Python?

(A) Encrypting source code with AES
(B) Translating an in-memory object hierarchy into a byte stream for storage or transmission
(C) Compiling Python into C++
(D) Compressing text with gzip

18. Which built-in module handles native binary object serialization and deserialization in Python?

(A) json
(B) pickle
(C) marshal
(D) csv

19. Which function in the pickle module serializes a Python object and writes it directly to an open binary file?

(A) pickle.dump(obj, file)
(B) pickle.dumps(obj)
(C) pickle.write(obj, file)
(D) pickle.save(obj, file)

20. Which function in the pickle module deserializes an object from an open binary file?

(A) pickle.load(file)
(B) pickle.loads(bytes)
(C) pickle.read(file)
(D) pickle.fetch(file)

21. Which keyword initiates an exception handling block in Python?

(A) catch
(B) try
(C) handle
(D) guard

22. Which keyword captures and handles exceptions raised inside a try block?

(A) catch
(B) except
(C) error
(D) rescue

23. When does the 'else' block attached to a try-except statement execute?

(A) Whenever an exception occurs
(B) Only if NO exception was raised in the try block
(C) Always after except
(D) Only if a finally block is omitted

24. When does the 'finally' block attached to a try statement execute?

(A) Only if an exception is raised
(B) Only if no exception is raised
(C) Always, regardless of whether exceptions were raised or caught
(D) Only during system shutdown

25. Which exception is raised when dividing an integer or float by zero?

(A) ArithmeticError
(B) ZeroDivisionError
(C) DivisionByZeroException
(D) MathError

26. Which exception is raised when trying to open a non-existent file for reading?

(A) FileMissingException
(B) FileNotFoundError (subclass of OSError)
(C) IOError
(D) PathError

27. Which exception is raised when an operation receives an argument of right type but inappropriate value (e.g. int('abc'))?

(A) TypeError
(B) ValueError
(C) KeyError
(D) IndexError

28. Which exception is raised when an operation is performed on an inappropriate or mismatched data type (e.g. '5' + 5)?

(A) ValueError
(B) TypeError
(C) AttributeError
(D) SyntaxError

29. Which keyword is used to explicitly trigger or raise an exception in Python?

(A) throw
(B) raise
(C) trigger
(D) emit

30. What base class should all user-defined custom exceptions inherit from in Python?

(A) BaseException
(B) Exception
(C) Error
(D) StandardError

31. Which built-in module provides regular expression pattern matching capabilities in Python?

(A) regex
(B) re
(C) pattern
(D) matcher

32. What does the regex metacharacter dot (.) match in standard mode?

(A) Any digit 0-9
(B) Any single character except a newline character '\n'
(C) Any punctuation symbol
(D) Any whitespace character

33. Which regex anchor metacharacter asserts the beginning of a line or string?

(A) $
(B) ^
(C) \b
(D) \A

34. Which regex anchor metacharacter asserts the end of a line or string?

(A) ^
(B) $
(C) \Z
(D) \e

35. What quantifier matches zero or more occurrences of the preceding regular expression element?

(A) +
(B) *
(C) ?
(D) {1,}

36. What quantifier matches one or more occurrences of the preceding element?

(A) *
(B) +
(C) ?
(D) {0,1}

37. What quantifier matches zero or one occurrence (optional) of the preceding element?

(A) *
(B) +
(C) ?
(D) {0,}

38. Which regex special sequence matches any ASCII or Unicode decimal digit [0-9]?

(A) \s
(B) \w
(C) \d
(D) \b

39. Which regex special sequence matches any alphanumeric word character [a-zA-Z0-9_]?

(A) \d
(B) \s
(C) \w
(D) \b

40. Which regex special sequence matches any whitespace character (space, tab, newline)?

(A) \w
(B) \d
(C) \s
(D) \W

41. What occurs when opening an existing file with mode 'w+'?

(A) File is opened for appending
(B) File is immediately truncated to 0 bytes and opened for both reading and writing
(C) Raises FileExistsError
(D) Read-only access

42. What is the difference between open('file.txt', 'r+') and open('file.txt', 'w+')?

(A) 'r+' opens for reading and writing without truncation; 'w+' truncates the file immediately to 0 bytes
(B) 'r+' creates the file if absent; 'w+' does not
(C) They are identical aliases
(D) 'w+' is for binary files only

43. What does file.seek(0, 2) accomplish on an open file object?

(A) Rewinds pointer to the very beginning of the file
(B) Positions the file pointer at the very end of the file
(C) Advances pointer by 2 bytes
(D) Raises ValueError

44. What are the valid integer values for the 'whence' argument in file.seek(offset, whence)?

(A) 0 (start of file), 1 (current pointer position), 2 (end of file)
(B) 1 (start), 2 (mid), 3 (end)
(C) Only 0 is permitted
(D) -1, 0, 1

45. What occurs when executing: with open('data.txt') as f: content = f.read(); print(f.closed)?

(A) Prints False
(B) Prints True (file is automatically closed upon leaving the with block)
(C) Raises an AttributeError
(D) None

46. What happens if an unhandled exception occurs inside a with open(...) block?

(A) The file remains open indefinitely leaking resources
(B) The context manager's __exit__ method executes and cleanly closes the file before propagating the exception
(C) The file is deleted from disk
(D) The OS crashes

47. Why is unpickling untrusted data from network sources considered a critical security vulnerability (RCE)?

(A) Pickle files can contain viruses in plain text
(B) Pickle allows executing arbitrary code via the __reduce__ method during deserialization
(C) Pickle uses too much CPU
(D) Pickle modifies file permissions

48. Which format is safer for serializing data between disparate languages and untrusted networks?

(A) pickle
(B) JSON (JavaScript Object Notation)
(C) marshal
(D) shelve

49. What is the output of:\ntry:\n x = 10 / 0\nexcept ZeroDivisionError:\n print('A', end='')\nelse:\n print('B', end='')\nfinally:\n print('C', end='')

(A) AC
(B) ABC
(C) BC
(D) C

50. What is the output of:\ntry:\n x = 10 / 2\nexcept ZeroDivisionError:\n print('A', end='')\nelse:\n print('B', end='')\nfinally:\n print('C', end='')

(A) BC
(B) ABC
(C) B
(D) C

51. What is the output of:\ndef f():\n try:\n return 1\n finally:\n return 2\nprint(f())

(A) 1
(B) 2 (The finally block's return statement overrides the try block's return!)
(C) 3
(D) SyntaxError

52. What is the output of:\nimport re\nres = re.search(r'\d+', 'User ID: 1042 in Dept 5')\nprint(res.group())

(A) 'User ID: 1042'
(B) '1042'
(C) '5'
(D) ['1042', '5']

53. What does re.findall(r'\d+', 'User ID: 1042 in Dept 5') return?

(A) ['1042']
(B) ['1042', '5']
(C) 1042
(D) Match object

54. What is the difference between re.match() and re.search() in Python?

(A) re.match() matches ONLY from the start of the string; re.search() scans the entire string
(B) re.search() is faster
(C) re.match() returns a list; re.search() returns a string
(D) They are identical

55. What is the output of: print(re.match(r'\d+', 'Score: 95'))?

(A) Match object for '95'
(B) None (because 'Score: 95' does not begin with digits)
(C) ['95']
(D) TypeError

56. What is the output of: print(re.search(r'\d+', 'Score: 95').group())?

(A) 'Score'
(B) '95'
(C) None
(D) Error

57. What does the regex quantifier *? represent (e.g. <.*?> vs <.*>)?

(A) Zero or one match
(B) Non-greedy (lazy) matching: matches as few characters as possible
(C) Greedy matching
(D) Case-insensitive matching

58. What does re.sub(r'\s+', '-', 'Python Data Structures') return?

(A) 'Python-Data-Structures'
(B) 'Python---Data---Structures'
(C) ['Python', 'Data', 'Structures']
(D) 'Python Data Structures'

59. What does re.split(r'[,;\s]+', 'apple, banana; orange grape') produce?

(A) ['apple', 'banana', 'orange', 'grape']
(B) ['apple', ',', 'banana', ';', 'orange', 'grape']
(C) 'apple-banana-orange-grape'
(D) TypeError

60. Why should regular expressions be compiled with re.compile(pattern) when used in loops?

(A) It reduces string length
(B) It pre-compiles pattern into a Pattern object, avoiding repetitive regex parsing and compilation overhead
(C) It runs in parallel threads
(D) It is required by PEP 8

61. What does the regex flag re.IGNORECASE (re.I) do?

(A) Ignores syntax errors in patterns
(B) Performs case-insensitive matching (e.g. [a-z] matches uppercase too)
(C) Ignores whitespace characters
(D) Skips punctuation

62. What does the regex flag re.DOTALL (re.S) modify?

(A) Makes all quantifiers greedy
(B) Enables the dot (.) metacharacter to match newline characters '\n' as well
(C) Matches only dots and periods
(D) Disables backslash escaping

63. What does the regex flag re.MULTILINE (re.M) modify?

(A) Matches multiple strings simultaneously
(B) Makes ^ and $ match at the start and end of each individual line within a multiline string
(C) Allocates multiple threads
(D) Matches emojis

64. What does match.groups() return on a Match object with pattern r'(\w+)-(\d+)' on 'Item-402'?

(A) ('Item-402',)
(B) ('Item', '402')
(C) ['Item', 402]
(D) {'Item': 402}

65. What syntax defines a named capturing group in Python regex?

(A) (?P<name>pattern)
(B) (?<name>pattern)
(C) (:name pattern)
(D) ($name=pattern)

66. What does match.groupdict() return when named capturing groups are used?

(A) A dictionary mapping group names to their matched substrings
(B) A tuple of names
(C) A list of indices
(D) None

67. What is the output of:\nimport os\nprint(os.path.join('dir', 'subdir', 'file.txt')) on Windows?

(A) 'dir/subdir/file.txt'
(B) 'dir\\subdir\\file.txt'
(C) 'dir.subdir.file.txt'
(D) TypeError

68. What does os.path.exists(path) return?

(A) File size
(B) True if the file or directory exists; False otherwise
(C) Creation timestamp
(D) File permissions

69. What does os.path.isfile(path) return?

(A) File size
(B) True if path exists and is a regular file; False otherwise
(C) File content
(D) Path name

70. What does shutil.rmtree(path) perform?

(A) Deletes a single empty directory
(B) Recursively deletes an entire directory tree and all its contents
(C) Moves a directory to recycle bin
(D) Renames a directory

71. What is the output of print('Hello\nWorld'.splitlines())?

(A) ['Hello\n', 'World']
(B) ['Hello', 'World']
(C) 'HelloWorld'
(D) ('Hello', 'World')

72. What does the expression raise ValueError('Invalid') from exc accomplish (PEP 3134)?

(A) Suppresses the original error
(B) Explicitly chains exceptions, setting __cause__ to preserve the original traceback context
(C) Clears the traceback
(D) Converts ValueError to exc

73. What occurs when catching multiple exceptions: except (TypeError, ValueError) as err:?

(A) Catches both TypeError and ValueError in a single unified handler block
(B) SyntaxError
(C) Catches only TypeError
(D) Creates a tuple of errors

74. Why is writing bare 'except:' without specifying an exception class considered an enterprise antipattern?

(A) SyntaxError in Python 3
(B) It catches BaseException, inadvertently intercepting SystemExit and KeyboardInterrupt (Ctrl+C)
(C) It slows down the CPU
(D) It crashes the interpreter

75. What is the recommended broad exception to catch when handling runtime failures: except Exception:?

(A) except BaseException:
(B) except Exception:
(C) except StandardError:
(D) except Error:

76. What type does file.read() return when a file is opened in binary mode: open('data.bin', 'rb')?

(A) str (Unicode string)
(B) bytes (Immutable sequence of 8-bit bytes)
(C) bytearray
(D) int array

77. What happens if you attempt to write a string to a file opened in binary mode: f.write('Hello')?

(A) String is encoded automatically to UTF-8
(B) TypeError: a bytes-like object is required, not 'str'
(C) ValueError
(D) Silent failure

78. What does open('file.txt', 'r', encoding='utf-8', errors='replace') do upon encountering corrupt bytes?

(A) Raises UnicodeDecodeError
(B) Replaces invalid byte sequences with the Unicode replacement character U+FFFD ()
(C) Skips the corrupt bytes
(D) Deletes the file

79. What type of regular expression engine does CPython's standard 're' module utilize?

(A) Deterministic Finite Automaton (DFA)
(B) Nondeterministic Finite Automaton (NFA) with backtracking
(C) Parsing Expression Grammar (PEG)
(D) Earley Parser

80. What severe performance pathology can occur in backtracking NFA regex engines when evaluating catastrophic nested quantifiers: (a+)+$ on 'aaaa...X'?

(A) Catastrophic Backtracking (Exponential O(2^N) time complexity causing ReDoS)
(B) Stack overflow in C
(C) Memory leak
(D) Infinite compilation

81. What does ReDoS stand for in cybersecurity?

(A) Regular Expression Denial of Service
(B) Recursive Directory Operating System
(C) Remote Exception Domain Service
(D) Redundant Disk Array Over Subnet

82. What does a positive lookahead assertion (?=pattern) do in regular expressions?

(A) Matches pattern and includes it in match result
(B) Asserts that pattern matches immediately following the current position without consuming characters
(C) Reverses search direction
(D) Creates a loop

83. What does a negative lookahead assertion (?!pattern) do?

(A) Asserts that pattern does NOT match immediately following current position without consuming characters
(B) Matches negative numbers
(C) Inverts character casing
(D) Deletes characters

84. What does a positive lookbehind assertion (?<=pattern) do?

(A) Asserts that pattern matches immediately preceding current position without consuming characters
(B) Scans from end of file
(C) Captures previous match
(D) Matches backwards in loop

85. What restriction exists for lookbehind assertions in Python's standard re module?

(A) Lookbehind patterns must have a fixed, known character width
(B) Cannot contain letters
(C) Cannot be negated
(D) No restrictions exist

86. What does non-capturing group (?:pattern) do compared to standard capturing group (pattern)?

(A) Groups regex tokens together for quantifiers without saving the match into a capturing slot
(B) Prevents matching
(C) Hides pattern from regex engine
(D) Matches only lowercase

87. What does the line buffering mode buffering=1 accomplish in open('log.txt', 'w', buffering=1)?

(A) Buffers exactly 1 byte
(B) Flushes buffer automatically on every newline character '\n' in text mode
(C) Disables buffering completely
(D) Allocates 1 MB buffer

88. How can binary unbuffered I/O be opened in Python?

(A) open('file.bin', 'wb', buffering=0)
(B) open('file.bin', 'wb', buffer=False)
(C) open('file.bin', 'wb', flush=True)
(D) buffering is always enabled

89. What is the difference between os.remove(path) and os.rmdir(path)?

(A) os.remove() deletes files; os.rmdir() deletes empty directories
(B) os.remove() deletes directories; os.rmdir() deletes files
(C) They are identical aliases
(D) os.remove() moves to recycle bin

90. What does the fileinput module in Python standard library facilitate?

(A) Graphical user input dialogs
(B) Iterating over lines from multiple input files or standard input sys.stdin seamlessly
(C) File compression
(D) Keyboard macro logging

91. What does tempfile.NamedTemporaryFile() provide?

(A) A permanent backup file
(B) A temporary file with a visible filesystem name that is automatically destroyed on close
(C) A virtual RAM disk
(D) A temporary directory

92. What does the traceback module allow developers to do in Python?

(A) Step through code line by line like a debugger
(B) Extract, format, and print detailed execution stack tracebacks programmatically
(C) Undo file modifications
(D) Rollback database commits

93. What does sys.exc_info() return inside an except block?

(A) A tuple of (type, value, traceback) for the exception currently being handled
(B) The name of the file that crashed
(C) The line number of the error only
(D) None

94. What occurs when calling warnings.warn('Warning Message', category=DeprecationWarning)?

(A) Program crashes immediately with SystemError
(B) Emits a non-fatal warning message to stderr that can be filtered via the warnings module
(C) Logs message to Windows Event Viewer
(D) Prints message in bold red

95. What does json.dumps(obj) return versus json.dump(obj, file)?

(A) json.dumps() returns a serialized JSON string; json.dump() writes serialized JSON directly to a file stream
(B) json.dump() is for strings; json.dumps() is for files
(C) They are identical aliases
(D) json.dumps() encrypts data

96. Why will json.dumps({'date': datetime.now()}) raise a TypeError?

(A) Dictionaries cannot contain dates
(B) datetime objects are not natively JSON serializable by default without a custom default serializer
(C) JSON does not support dictionaries
(D) Date is a reserved keyword

97. What is the purpose of the default parameter in json.dumps(obj, default=custom_serializer)?

(A) Provides a fallback JSON string if parsing fails
(B) Specifies a callable to convert non-serializable objects into JSON-compatible primitives
(C) Sets default indentation
(D) Sorts dictionary keys

98. What does re.subn(pattern, repl, string) return in Python?

(A) A tuple containing: (new_string, number_of_substitutions_made)
(B) Only the number of substitutions
(C) A list of strings
(D) The matched spans

99. What does the regex special sequence \b represent?

(A) Backspace character
(B) Word boundary assertion (transition between word character \w and non-word character \W)
(C) Binary character
(D) Beginning of buffer

100. What does re.escape(string) perform in Python?

(A) Escapes all characters that might be interpreted as regex metacharacters, allowing safe literal matching
(B) Converts string to ASCII
(C) Strips quotes from string
(D) Removes backslashes

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain File Handling in Python. Detail open() modes ('r', 'w', 'a', 'x', '+', 'b'), buffer flushing, and why Context Managers (with) are mandatory in production.

Q2. Explain File Pointer Positioning using seek() and tell(). Detail the 'whence' parameter (0, 1, 2) in text vs binary modes with practical file manipulation.

Q3. Explain the os and os.path Modules in Python. Demonstrate directory navigation, directory creation, path joining, listing, file metadata, and shutil operations.

Q4. Explain Modern Object-Oriented Path Management using the pathlib Module (PEP 428). Compare pathlib.Path with legacy os.path.

Q5. Explain Object Serialization using the pickle Module. Detail pickle.dump(), pickle.load(), the pickle protocol versions, and security risks of unpickling untrusted data.

Q6. Explain Exception Handling Architecture in Python: try, except, else, finally, and the Built-in Exception Hierarchy (BaseException vs Exception).

Q7. Explain Exception Raising, Exception Chaining (raise ... from ...), and Custom User-Defined Exception Classes in Python with domain models.

Q8. Explain the re Module Architecture in Python. Detail regex compilation, pattern matching lifecycle, and compare match(), search(), findall(), finditer(), and sub().

Q9. Explain Regular Expression Metacharacters, Quantifiers, and Character Classes in Python. Contrast Greedy versus Non-Greedy (Lazy) Matching with illustrative HTML parsing.

Q10. Explain Capturing Groups, Non-Capturing Groups, Named Groups, and Backreferences in Python Regular Expressions with practical code examples.

Q11. Explain Lookahead and Lookbehind Assertions (Zero-Width Assertions) in Python Regular Expressions: (?=...), (?!...), (?<=...), and (?<!...).

Q12. Implement a Robust CSV and JSON File Data Processing Utility in Python with schema validation, type parsing, and transactional file writing.

Q13. Implement a Web Server Log File Parser in Python using Regular Expressions, extracting IP addresses, HTTP status codes, endpoints, and error distributions.

Q14. Implement an Enterprise Configuration File Manager in Python supporting .env parsing, variable expansion, type coercion, and comments.

Q15. Implement a Credit Card and Phone Number Validator in Python using Luhn's Algorithm and Regular Expressions.

Q16. Implement a Sensitive Data Sanitizer and PII Redactor in Python using Regular Expressions to mask emails, phone numbers, and SSNs in unstructured text.

Q17. Implement a File Deduplication and Checksum Verification Tool in Python using hashlib (SHA-256) and os.walk to detect identical files across directories.

Q18. Implement an Automated Log Rotation and Compression Daemon Simulation in Python using gzip, os, and time.

Q19. Implement an INI-Style Configuration Parser in Python using Regular Expressions without external libraries.

Q20. Implement a Safe Atomic File Writer in Python using Context Managers, ensuring files are never corrupted by mid-write crashes.

Q21. Implement a Markdown to HTML Converter in Python using Regular Expressions for headers, bold, italics, links, and code blocks.

Q22. Implement an Asynchronous Exception-Resilient Web Crawler Queue Simulation in Python with retry budgets and exponential backoff.

Q23. Implement a Data File Anonymization Utility in Python using Regex and CSV to mask names and IP addresses in security audit logs.

Q24. Implement a Custom Binary Protocol Pack/Unpack Serializer in Python using struct module for fixed-width packet serialization.

Q25. Implement a Multi-Threaded File Downloader Simulator in Python using concurrent.futures and exception handling for failed chunks.

Q26. Implement a File Integrity Verification Utility in Python that computes MD5, SHA-1, and SHA-256 digests simultaneously in a single file pass.

Q27. Implement a Universal File Format Detector in Python using Magic Numbers (File Signatures) by inspecting the first 8 header bytes.

Q28. Implement a Syntax Highlighter Lexer in Python using Regular Expressions to tokenize Python source code into Keywords, Identifiers, Numbers, Strings, and Operators.

Q29. Implement an Exception-Safe Transactional Database File Journaling System in Python using write-ahead logs (WAL).

Q30. Implement a Complete Email, URL, and Date Format Extractor in Python using Regular Expressions with rigorous boundary constraints.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[B]	**Q21:**	[B]	**Q41:**	[B]	**Q61:**	[B]	**Q81:**	[A]
Q2:	[B]	**Q22:**	[B]	**Q42:**	[A]	**Q62:**	[B]	**Q82:**	[B]
Q3:	[B]	**Q23:**	[B]	**Q43:**	[B]	**Q63:**	[B]	**Q83:**	[A]
Q4:	[C]	**Q24:**	[C]	**Q44:**	[A]	**Q64:**	[B]	**Q84:**	[A]
Q5:	[C]	**Q25:**	[B]	**Q45:**	[B]	**Q65:**	[A]	**Q85:**	[A]
Q6:	[B]	**Q26:**	[B]	**Q46:**	[B]	**Q66:**	[A]	**Q86:**	[A]
Q7:	[B]	**Q27:**	[B]	**Q47:**	[B]	**Q67:**	[B]	**Q87:**	[B]
Q8:	[B]	**Q28:**	[B]	**Q48:**	[B]	**Q68:**	[B]	**Q88:**	[A]
Q9:	[C]	**Q29:**	[B]	**Q49:**	[A]	**Q69:**	[B]	**Q89:**	[A]
Q10:	[B]	**Q30:**	[B]	**Q50:**	[A]	**Q70:**	[B]	**Q90:**	[B]
Q11:	[A]	**Q31:**	[B]	**Q51:**	[B]	**Q71:**	[B]	**Q91:**	[B]
Q12:	[B]	**Q32:**	[B]	**Q52:**	[B]	**Q72:**	[B]	**Q92:**	[B]
Q13:	[A]	**Q33:**	[B]	**Q53:**	[B]	**Q73:**	[A]	**Q93:**	[A]
Q14:	[B]	**Q34:**	[B]	**Q54:**	[A]	**Q74:**	[B]	**Q94:**	[B]
Q15:	[B]	**Q35:**	[B]	**Q55:**	[B]	**Q75:**	[B]	**Q95:**	[A]
Q16:	[B]	**Q36:**	[B]	**Q56:**	[B]	**Q76:**	[B]	**Q96:**	[B]
Q17:	[B]	**Q37:**	[C]	**Q57:**	[B]	**Q77:**	[B]	**Q97:**	[B]
Q18:	[B]	**Q38:**	[C]	**Q58:**	[A]	**Q78:**	[B]	**Q98:**	[A]
Q19:	[A]	**Q39:**	[C]	**Q59:**	[A]	**Q79:**	[B]	**Q99:**	[B]
Q20:	[A]	**Q40:**	[C]	**Q60:**	[B]	**Q80:**	[A]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

SECTION B: DETAILED SOLUTIONS FOR SUBJECTIVE QUESTIONS Modes ('r', 'w', 'a', 'x', '+', 'b'), buffer flushing, and why Context Managers (with) are manda

```
# Cleanup test artifact
if os.path.exists(filename):
os.remove(filename)

file_io_demonstration()
```

5. Complexity Analysis:
- File Open / Close Overhead: O(1) OS syscall.
- Line Iteration Time: O(N) where N is file size; Space: O(1) streaming buffer memory.

Solution to Question 2: Explain File Pointer Positioning using seek() and tell(). Detail the 'whence' parameter (0, 1, 2) in text vs binary modes with practical file

```
f.seek(-2, os.SEEK_CUR)
print(f"After seek(-2, SEEK_CUR): Pointer is at {f.tell()} | Byte: {f.read(1)[0]}")

# Seek relative to end of file (whence = 2)
f.seek(-2, os.SEEK_END)
print(f"After seek(-2, SEEK_END): Pointer is at {f.tell()} | Remaining: {list(f.read())}")

if os.path.exists(bin_file):
os.remove(bin_file)

demonstrate_seek_and_tell()

4. Complexity Analysis:
- seek() / tell() Time Complexity: O(1) direct OS lseek syscall.
- Space Complexity: O(1) auxiliary frame memory.
```

>>P Solution Statement Question 3: Explain the os and os.path Modules in Python. Demonstrate directory navigation, directory creation, path joining, listing, file metadata, a

```
# 4. Copying file via shutil
backup_file = os.path.join(target_dir, "sys_backup.log")
shutil.copy2(sample_file, backup_file)
print(f"Directory Contents: {os.listdir(target_dir)}")

# 5. Clean teardown via shutil.rmtree
shutil.rmtree(base_dir)
print(f"Successfully cleaned sandbox directory: {base_dir}")

filesystem_pipeline_demo()
```

3. Complexity Analysis:
- Path operations: O(1) string concatenations.
- rmtree Time: O(Total Inodes) recursive traversal.
- Space Complexity: O(Depth) directory recursion.

>> Solution to Question 4: Explain Modern Object-Oriented Path Management using the pathlib Module (PEP 428), Compare pathlib.Path with legacy os.path.

```
print(f"Read Content via read_text(): {content}")

# 4. Path globbing
found_files = list(sandbox.rglob("*.json"))
print(f"Glob Found Files: {found_files}")

# Cleanup
target_file.unlink() # Delete file
sub_dir.rmdir()
(sandbox / "config").rmdir()
sandbox.rmdir()
print("Cleaned pathlib sandbox successfully.")

pathlib_modern_demo()
```

3. Complexity Analysis:
- Path Operations: O(1) object allocations.
- File Read/Write: O(N) where N is file size.

>> Solution to Question 5: Explain Object Serialization using the pickle Module. Detail pickle.dump(), pickle.load(), the pickle protocol versions, and security risks of

```
def test_pickling_lifecycle(filename="model.pkl"):
orig_config = NeuralNetConfig([128, 64, 10], 0.001)

# 1. Serialize to binary file
with open(filename, "wb") as f:
pickle.dump(orig_config, f, protocol=pickle.HIGHEST_PROTOCOL)

print(f"Serialized object to {filename} ({os.path.getsize(filename)} bytes)")

# 2. Deserialize from binary file
with open(filename, "rb") as f:
restored_config = pickle.load(f)

print(f"Restored Object: {restored_config}")
print(f"Weights Intact: {restored_config.weights}")
print(f"Deep Clone Check (is): {restored_config is not orig_config}")

if os.path.exists(filename):
os.remove(filename)

test_pickling_lifecycle()

4. Complexity Analysis:
- Serialization Time: O(Total Nodes) recursive object graph traversal.
- Space Complexity: O(Byte Stream Length).
```

>>Solution to Question 6: Explain Exception Handling Architecture in Python: try, except, else, finally, and the Built-in Exception Hierarchy (BaseException vs Exc

```
result = None

try:
n = float(numerator)
d = float(denominator)
result = n / d
except ZeroDivisionError as ex:
print(f"[EXCEPT] Mathematical Error: Cannot divide by zero ({ex})")
except ValueError as ex:
print(f"[EXCEPT] Conversion Error: Non-numeric operand passed ({ex})")
except Exception as ex:
print(f"[EXCEPT] Unexpected Failure: {type(ex).__name__} -> {ex}")
else:
print(f"[ELSE] Division completed successfully! Result = {result:.4f}")
finally:
print("[FINALLY] Cleanup routine completed. Execution channel closed.")

return result

# Verification of all execution paths
robust_division_service(100, 4) # Triggers try -> else -> finally
robust_division_service(100, 0) # Triggers try -> except (ZeroDivisionError) -> finally
robust_division_service("abc", 5) # Triggers try -> except (ValueError) -> finally

4. Complexity Analysis:
- Happy Path Time: Zero cost (zero-cost exceptions in Python 3.11+).
- Exception Path Time: O(Stack Depth) traceback construction.
```

>> Solution to Question 7: Explain Exception Raising, Exception Chaining (raise ... from ...), and Custom User-Defined Exception Classes in Python with domain r

```
pass

class InsufficientFundsError(BankingDomainError):
def __init__(self, requested: float, available: float):
super().__init__(f"Transaction declined: Requested ${requested:,.2f}, Available:
${available:,.2f}")
self.requested = requested
self.available = available

class DailyWithdrawalLimitExceededError(BankingDomainError):
pass

def process_atm_withdrawal(current_balance: float, requested_amount: float, daily_limit:
float):
if requested_amount > daily_limit:
raise DailyWithdrawalLimitExceededError(f"Amount ${requested_amount} exceeds daily limit of
${daily_limit}")
if requested_amount > current_balance:
raise InsufficientFundsError(requested_amount, current_balance)
return current_balance - requested_amount

# Demonstration
try:
process_atm_withdrawal(current_balance=250.0, requested_amount=500.0, daily_limit=1000.0)
except InsufficientFundsError as err:
print(f"Caught Specific Domain Exception: {err}")
print(f"Deficit Amount: ${err.requested - err.available:.2f}")

5. Complexity Analysis:
- Exception Instantiation: O(1) object allocation.
- Space Complexity: O(Depth) traceback frames.
```

>> Solution to Question 8: Explain the re Module Architecture in Python. Detail regex compilation, pattern matching lifecycle, and compare match(), search(), findall(), and finditer() methods.

- re.finditer(pat, str): Returns an iterator yielding Match objects for all non-overlapping matches (Memory efficient!).
- re.sub(pat, repl, str): Searches and replaces all matches with repl.

3. Python Code Demonstration:

```
import re

log_data = "IP: 192.168.1.10 - User: admin - Status: 200\nIP: 10.0.0.5 - User: guest - Status: 404"

# Pre-compile regex for efficiency
ip_pattern = re.compile(r'IP:\s+(\d{1,3}(?:\.\d{1,3}){3})')

# 1. search()
first_match = ip_pattern.search(log_data)
print(f"re.search first match: {first_match.group(1)}")

# 2. findall()
all_ips = ip_pattern.findall(log_data)
print(f"re.findall list: {all_ips}")

# 3. finditer() with span coordinates
print("re.finditer stream:")
for m in ip_pattern.finditer(log_data):
print(f" Matched IP: {m.group(1):<15} at Span: {m.span(1)}")

# 4. sub() data masking
masked_log = ip_pattern.sub("IP: [REDACTED]", log_data)
print(f"\nMasked Log:\n{masked_log}")
```

4. Complexity Analysis:
- Pattern Compilation Time: O(P) where P is pattern length.
- Search Time: O(N) linear time for non-pathological regular expressions.

>>Solution to Question 9: Explain Regular Expression Metacharacters, Quantifiers, and Character Classes in Python. Contrast Greedy versus Non-Greedy (Lazy

3. The Classic HTML Parsing Dilemma:
Consider parsing HTML tags in text: ' Title ' with pattern '<.*>':
- Greedy '<.*>': Matches from the very first '<' all the way to the very last '>'!
Result: ' Title ' (A single massive catastrophic match!).
- Lazy '<.*?>': Matches from each '<' up to the nearest '>'.
Result: ' ', '', '', ' ' (Correctly tokenizes individual tags!).

4. Python Code Demonstration:

```
import re

html_snippet = "<div><p>Paragraph 1</p><span>Caption</span></div>"

greedy_pat = re.compile(r'<.*>')
lazy_pat = re.compile(r'<.*?>')

greedy_matches = greedy_pat.findall(html_snippet)
lazy_matches = lazy_pat.findall(html_snippet)

print(f"Original HTML: '{html_snippet}'")
print(f"\nGreedy (<.*>): Matches {len(greedy_matches)} item: {greedy_matches}")
print(f"Non-Greedy (<.*?>): Matches {len(lazy_matches)} items: {lazy_matches}")

# Extracting tag content via lazy capture
content_pat = re.compile(r'<p>(.*?)</p>')
print(f"Extracted Paragraph Content: {content_pat.findall(html_snippet)}")
```

5. Complexity Analysis:
- Time Complexity: O(N) single scan; Greedy backtracking may degrade to O(N^2) if misconfigured.
- Space Complexity: O(Matches) memory.

>> Solution to Question 10: Explain Capturing Groups, Non-Capturing Groups, Named Groups, and Backreferences in Python Regular Expressions with practical c

```
url_regex = re.compile(
r'^(?P<scheme>https?)://'
r'(?P<host>[a-zA-Z0-9.-]+)'
r'(?::(?P<port>\d+))?'
r'(?P<path>/[a-zA-Z0-9/_.-]*)?$'
)

sample_url = "https://api.university.edu:8443/v1/students"
m = url_regex.match(sample_url)

print("Parsed URL Named Groups (groupdict):")
for k, v in m.groupdict().items():
print(f" {k:<10}: {v}")

# 2. Backreference Demonstration (Duplicate Words)
prose = "Paris in the the spring was was beautiful."
dup_regex = re.compile(r' (?P<word>\w+)\s+(?P=word) ', re.IGNORECASE)

print("
Duplicate Word Detection using Backreferences:")
for dup in dup_regex.finditer(prose):
print(f" Found duplicate: '{dup.group(0)}' (Repeated Word: '{dup.group('word')}')")
```

4. Complexity Analysis:
- Regex Evaluation Time: O(N) linear time.
- Space Complexity: O(G) where G is number of captured groups.

>>Solution to Question 11 Explain Lookahead and Lookbehind Assertions (Zero-Width Assertions) in Python Regular Expressions: (?=...), (?!...), (?<=...), and (?<

```
passwords = ["weak", "NoDigits!", "validPass1@", "short1!A", "ValidPass2026#"]

print("Password Validation via Multiple Lookahead Assertions:")
for pwd in passwords:
is_valid = bool(password_validator.match(pwd))
status = "[STRONG]" if is_valid else "[REJECTED]"
print(f" {status:<12} '{pwd}'")

# Lookbehind currency extraction
price_text = "Standard: $45.00, Premium: ?90.00, Discount: $25.50"
dollar_amounts = re.findall(r'(?<=\$)\d+\.\d{2}', price_text)
print(f"\nExtracted Dollar Amounts via Lookbehind (?<=\$): {dollar_amounts}")

4. Complexity Analysis:
- Time Complexity: O(N * Assertions) evaluated at string boundary.
- Space Complexity: O(1) zero-width match state.
```

>> Solution to Question 12: Implement a Robust CSV and JSON File Data Processing Utility in Python with schema validation, type parsing, and transactional file w

```
"Departments": {}
}
for dept, salaries in dept_salaries.items():
summary["Departments"][dept] = {
"Headcount": len(salaries),
"TotalPayroll": sum(salaries),
"AverageSalary": round(sum(salaries) / len(salaries), 2)
}

# Atomic file writing via temporary file replacement
dir_name = os.path.dirname(output_json_path) or "."
with tempfile.NamedTemporaryFile("w", dir=dir_name, delete=False, encoding="utf-8") as tf:
json.dump(summary, tf, indent=2)
temp_name = tf.name

os.replace(temp_name, output_json_path) # Atomic OS rename!
return summary

# Test execution with mock CSV
mock_csv = "emp_test.csv"
mock_json = "summary_test.json"

with open(mock_csv, "w", newline="", encoding="utf-8") as f:
writer = csv.writer(f)
writer.writerow(["ID", "Name", "Department", "Salary"])
writer.writerow(["101", "Aarav", "Engineering", "95000.0"])
writer.writerow(["102", "Priya", "Engineering", "105000.0"])
writer.writerow(["103", "Meera", "Marketing", "72000.0"])

res = DataPipelineProcessor.process_and_summarize(mock_csv, mock_json)
print("Aggregated JSON Output:", json.dumps(res, indent=2))

# Cleanup
for p in (mock_csv, mock_json):
if os.path.exists(p): os.remove(p)
```

3. Complexity Analysis:
- Time Complexity: O(N) single linear pass over CSV records.
- Space Complexity: O(D) where D is number of unique departments.

>>Solution to Question 13: Implement a Web Server Log File Parser in Python using Regular Expressions, extracting IP addresses, HTTP status codes, endpoints

```
class WebLogParser:
LOG_REGEX = re.compile(
r'^(?P<ip>\S+)\s+\S+\s+\S+\s+'
r'\[(?P<timestamp>[^\]]+)\]\s+'
r'"(?P<method>[A-Z]+)\s+(?P<endpoint>\S+)\s+HTTP/[0-9.]+"\s+'
r'(?P<status>\d{3})\s+(?P<bytes>\S+)'
)

@classmethod
def parse_logs(cls, log_lines: list) -> dict:
status_counts = Counter()
ip_counts = Counter()
parsed_entries = []

for line in log_lines:
match = cls.LOG_REGEX.match(line.strip())
if match:
data = match.groupdict()
status_counts[data["status"]] += 1
ip_counts[data["ip"]] += 1
parsed_entries.append(data)

return {
"TotalProcessed": len(parsed_entries),
"StatusDistribution": dict(status_counts),
"TopIPs": ip_counts.most_common(2)
}

# Verification
sample_logs = [
'192.168.1.10 - - [26/Oct/2026:10:00:01 +0000] "GET /api/v1/users HTTP/1.1" 200 4520',
'10.0.0.5 - - [26/Oct/2026:10:00:05 +0000] "POST /login HTTP/1.1" 401 120',
'192.168.1.10 - - [26/Oct/2026:10:00:10 +0000] "GET /favicon.ico HTTP/1.1" 404 220',
'172.16.0.2 - - [26/Oct/2026:10:00:15 +0000] "GET /index.html HTTP/1.1" 200 8920'
]

report = WebLogParser.parse_logs(sample_logs)
print("Web Log Analysis Report:", report)

3. Complexity Analysis:
- Time Complexity: O(L * C) where L is lines and C is characters per line.
- Space Complexity: O(L) entries and frequency maps.
```

Solution to Question 14:

>>> Implement an Enterprise Configuration File Manager in Python supporting .env parsing, variable expansion, type coercion, and comme

```
class EnvConfigManager:
@staticmethod
def parse_env_string(env_text: str) -> dict:
config = {}
for line in env_text.splitlines():
line = line.strip()
# Ignore empty lines and comments
if not line or line.startswith('#'):
continue

if '=' in line:
key, val = line.split('=', 1)
key = key.strip()
val = val.strip()

# Strip inline comment
if '#' in val:
val = val.split('#', 1)[0].strip()

# Strip matching surrounding quotes
if (val.startswith('"') and val.endswith('"')) or (val.startswith("'") and val.endswith("'")):
val = val[1:-1]

# Type coercion
if val.lower() == 'true': val = True
elif val.lower() == 'false': val = False
elif val.isdigit(): val = int(val)

config[key] = val
return config

# Verification
raw_env = """
# Server Runtime Configuration
PORT=8080 # Production port
DEBUG=false
DATABASE_URL="postgres://user:secret@localhost:5432/app_db"
WORKERS=4
"""

parsed_cfg = EnvConfigManager.parse_env_string(raw_env)
print("Parsed Environment Configuration:")
for k, v in parsed_cfg.items():
print(f" {k:<15} -> {v} ({type(v).__name__})")
```

3. Complexity Analysis:
- Time Complexity: O(N) linear scan over lines.
- Space Complexity: O(K) dictionary storage.

Solution to Question 15:

>> Problem Statement: Implement a Credit Card and Phone Number Validator in Python using Luhn's Algorithm and Regular Expressions.

1. Problem Context:
Verify credit card numbers by combining regex structural format validation (16 digits separated by spaces/hyphens) with Luhn's Algorithm (Modulus 10 checksum).

2. Luhn's Algorithm:
1. Double every second digit from right to left.
2. If doubling yields >= 10, sum the digits (e.g. 14 -> 1 + 4 = 5).
3. Sum all modified and unmodified digits.
4. If sum % 10 == 0, the card number is mathematically valid.

3. Python Implementation:

```
import re

class PaymentValidator:
CARD_REGEX = re.compile(r'^(\d{4}[- ]?){3}\d{4}$')

@staticmethod
def luhn_check(card_number_str: str) -> bool:
digits = [int(ch) for ch in card_number_str if ch.isdigit()]
if len(digits) != 16:
return False

checksum = 0
reverse_digits = digits[::-1]

for idx, d in enumerate(reverse_digits):
if idx % 2 == 1:
doubled = d * 2
checksum += (doubled - 9) if doubled > 9 else doubled
else:
checksum += d

return checksum % 10 == 0

@classmethod
def validate_card(cls, card_str: str) -> bool:
if not cls.CARD_REGEX.match(card_str):
return False
return cls.luhn_check(card_str)

# Verification
sample_card = "4532-0150-1234-5678"
is_valid_format = bool(PaymentValidator.CARD_REGEX.match(sample_card))
is_valid_luhn = PaymentValidator.luhn_check(sample_card)

print(f"Card: '{sample_card}'")
print(f"Format Regex Valid? {is_valid_format}")
print(f"Luhn Algorithm Valid? {is_valid_luhn}")
```

4. Complexity Analysis:
- Time Complexity: O(1) constant time (exactly 16 digits).
- Space Complexity: O(1) integer registers.

---

4. Group files with matching SHA-256 digests.

2. Python Implementation:

```
import hashlib
import os
from collections import defaultdict

def find_duplicate_files(root_folder: str) -> dict:
size_map = defaultdict(list)

# Pass 1: Group by file size
for dirpath, _, filenames in os.walk(root_folder):
for fn in filenames:
fp = os.path.join(dirpath, fn)
try:
sz = os.path.getsize(fp)
size_map[sz].append(fp)
except OSError:
pass

duplicates = defaultdict(list)

# Pass 2: Hash only candidate files with size collision
for sz, files in size_map.items():
if len(files) > 1 and sz > 0:
for fp in files:
h = hashlib.sha256()
try:
with open(fp, "rb") as f:
while chunk := f.read(65536):
h.update(chunk)
digest = h.hexdigest()
duplicates[digest].append(fp)
except OSError:
pass

# Filter only genuine duplicates
return {k: v for k, v in duplicates.items() if len(v) > 1}

# Demonstration on existing python syllabus directory
target = "C:/FINAL PrOJECT/cse unit folder/python"
dups = find_duplicate_files(target)
print(f"Found {len(dups)} duplicate content groups in '{target}'.")
```

3. Complexity Analysis:
- Time Complexity: O(Files) size checks + O(Candidate Bytes) hashing.
- Space Complexity: O(Files) path metadata.

Solution to Question 18:

>>> Problem Statement: Implement an Automated Log Rotation and Compression Daemon Simulation in Python using gzip, os, and time.

Unattended server logs grow indefinitely, eventually filling disk drives.
Log Rotation rotates active logs once they exceed a size threshold (e.g., 5 MB) by renaming them with timestamps, compressing them with gzip (.gz), and maintaining a maximum backlog of archives.

2. Python Implementation:

```
import gzip
import os
import shutil
import time

class LogRotator:
def __init__(self, log_path: str, max_bytes=1024, max_backups=3):
self.log_path = log_path
self.max_bytes = max_bytes
self.max_backups = max_backups

def rotate_if_needed(self):
if not os.path.exists(self.log_path):
return

if os.path.getsize(self.log_path) >= self.max_bytes:
timestamp = int(time.time())
archive_name = f"{self.log_path}.{timestamp}.gz"

# Compress active log to .gz
with open(self.log_path, 'rb') as f_in:
with gzip.open(archive_name, 'wb') as f_out:
shutil.copyfileobj(f_in, f_out)

# Truncate active log file to 0 bytes
with open(self.log_path, 'w') as f:
pass

print(f"[ROTATED] Compressed active log to: {archive_name}")

# Verification
test_log = "service.log"
with open(test_log, "w") as f:
f.write("A" * 2000) # Exceeds 1024 bytes

rotator = LogRotator(test_log, max_bytes=1024)
rotator.rotate_if_needed()

# Cleanup
if os.path.exists(test_log): os.remove(test_log)
for f in os.listdir("."):
if f.startswith("service.log."): os.remove(f)
```

3. Complexity Analysis:
- Compression Time: O(File Size) streaming gzip deflation.
- Space Complexity: O(64 KB) streaming buffer.

Solution to Question 19: Implement an INI-Style Configuration Parser in Python using Regular Expressions without external libraries.

3. Complexity Analysis:
- Time Complexity: O(Lines) regex matching.
- Space Complexity: O(Sections * Keys) dictionary.

Solution to Question 20:

>>> Implement a Safe Atomic File Writer in Python using Context Managers, ensuring files are never corrupted by mid-write crashes.

```
# Demonstration
dest = "critical_database.json"
with AtomicFileWriter(dest) as f:
f.write('{"status": "stable", "records": 1000}')

with open(dest, "r") as f:
print("Verified file content:", f.read())

if os.path.exists(dest): os.remove(dest)
```

3. Complexity Analysis:
- Commit Overhead: O(1) atomic directory entry swap.
- Space Complexity: O(File Size) temporary storage.

Solution to Question 21: >>> Implement a Markdown to HTML Converter in Python using Regular Expressions for headers, bold, italics, links, and code blocks.

```
# Verification
sample_md = """# Python Programming
## Unit 6: Regular Expressions
Learn **Python** at [Google](https://google.com) using `re` module!"""

html_out = MarkdownParser.parse_markdown(sample_md)
print("Converted HTML:\n", html_out)
```

3. Complexity Analysis:
- Time Complexity: O(N) regex line substitution.
- Space Complexity: O(N) HTML string buffer.

>> Solution to Question 22: Implement an Asynchronous Exception-Resilient Web Crawler Queue Simulation in Python with retry budgets and exponential backoff.

>>Solution to Question 28: Implement a Data File Anonymization Pipeline Using Python's Built-in Modules and Regular Expressions

2. Python Implementation:

```
import struct

class NetworkPacket:
# Header format: Magic (2B), Version (1B), PacketType (1B), Sequence (4B), PayloadLen (2B)
HEADER_FORMAT = ">HBB I H"
HEADER_SIZE = struct.calcsize(HEADER_FORMAT) # 10 bytes

@classmethod
def pack(cls, seq_num: int, payload: str) -> bytes:
magic = 0xABCD
version = 1
pkt_type = 2 # DATA
encoded_payload = payload.encode('utf-8')
payload_len = len(encoded_payload)

header = struct.pack(cls.HEADER_FORMAT, magic, version, pkt_type, seq_num, payload_len)
return header + encoded_payload

@classmethod
def unpack(cls, raw_bytes: bytes) -> tuple:
header_bytes = raw_bytes[:cls.HEADER_SIZE]
magic, ver, p_type, seq, p_len = struct.unpack(cls.HEADER_FORMAT, header_bytes)
payload = raw_bytes[cls.HEADER_SIZE:cls.HEADER_SIZE + p_len].decode('utf-8')
return {"Magic": hex(magic), "Seq": seq, "Payload": payload}

# Demonstration
packet_bytes = NetworkPacket.pack(1042, "Telemetry Sensor Ping")
print(f"Packed Binary Packet: {packet_bytes} ({len(packet_bytes)} bytes)")

decoded = NetworkPacket.unpack(packet_bytes)
print(f"Unpacked Packet: {decoded}")
```

3. Complexity Analysis:
- Pack/Unpack Time: O(1) fixed-size header struct mapping.
- Space Complexity: O(Payload Length) byte array.

>> Solution to Question 25: Implement a Multi-Threaded File Downloader Simulator in Python using concurrent.futures and exception handling for failed chunks.

```
  pass
  data = f"[DATA_CHUNK_{chunk_id}]"
  return chunk_id, data

  def parallel_file_assembler(total_chunks=5):
  chunks = {}
  with ThreadPoolExecutor(max_workers=3) as executor:
  future_map = {executor.submit(download_chunk, i): i for i in range(1, total_chunks + 1)}
  for future in as_completed(future_map):
  c_id, content = future.result()
  chunks[c_id] = content
  print(f"Downloaded chunk #{c_id}")

  # Assemble in order
  final_stream = "".join(chunks[i] for i in range(1, total_chunks + 1))
  return final_stream

  assembled = parallel_file_assembler(5)
  print(f"Final Assembled Stream: '{assembled}'")
```

3. Complexity Analysis:
- Time Complexity: O(N / Workers) parallel execution.
- Space Complexity: O(N) assembled buffer.

>>P Solution to Question 26. Implement a File Integrity Verification Utility in Python that computes MD5, SHA-1, and SHA-256 digests simultaneously in a single file

```
# Create test file
test_file = "integrity_test.dat"
with open(test_file, "wb") as f:
f.write(b"Engineering Python Cryptographic Verification 2026")

hashes = multi_hash_stream(test_file)
print("Computed File Checksums:")
for alg, h in hashes.items():
print(f" {alg:<8}: {h}")

if os.path.exists(test_file): os.remove(test_file)
```

3. Complexity Analysis:
- Time Complexity: O(N) single I/O read pass over N bytes.
- Space Complexity: O(1) constant 64 KB memory chunk buffer.

>> Solution to Question 27: Implement a Universal File Format Detector in Python using Magic Numbers (File Signatures) by inspecting the first 8 header bytes.

3. Complexity Analysis:
- Identification Time: O(1) prefix byte comparison.
- Space Complexity: O(1) memory.

Solution to Question 28:

>>> Implement a Syntax Highlighter Lexer in Python using Regular Expressions to tokenize Python source code into Keywords, Identifiers,

Solution to Question 29:

```
>>> # Implement an Exception-Safe Transactional Database File Journaling System in Python using write-ahead logs (WAL),
```

Solution to Question 30: Implement a Complete Email, URL, and Date Format Extractor in Python using Regular Expressions with rigorous boundary constraints